

Diego Troy Lopez¹, Thomas B Reddington², and Robert Pahle¹

¹Research Technology Services, New York University

²Department of Computer Science, New York University

June 05, 2025

Abstract

Science DMZs are specialized networks designed for high performance scientific applications. Traditional security controls such as hardware firewalls are unsuitable in this context due to their throughput limitations. Network Intrusion Detection Systems (NIDS) analyze traffic out-of-line through mirroring techniques, which offers security value without compromising performance. Traditional NIDS leverage Switchport Analyzer (SPAN) to mirror Internet-bound traffic to the NIDS at the border of a Science DMZ. Capturing lateral movement traffic after an intrusion is challenging due to the cost of NIDS and physical connection requirements. While Encapsulated Remote SPAN allows mirroring traffic over IP, it is often seen as impractical due to the challenge of controlling its bandwidth usage. This work leverages Deep Learning techniques to develop an intelligent, dynamic NIDS capable of real-time lateral movement detection while preserving network performance. Our proposed system combines SPAN for mirroring Internet-bound traffic with filtered ERSPAN for selective East-West traffic mirroring. An Autoencoder detects anomalous behavior by comparing traffic flows to learned normal patterns. When reconstruction errors exceed a threshold, the system dynamically adjusts access control lists to mirror East-West traffic from only these potentially compromised hosts. In a functional evaluation, the system successfully detected an attack chain and captured intra-VLAN lateral movement. During an operational evaluation in a production Science DMZ environment, the system demonstrated minimal control plane overhead with an acceptable bandwidth usage. This first-of-a-kind approach enhances security, preserves operational integrity, and yields significant cost savings over traditional NIDS deploys that provide similar coverage.

Right Place, Right Time: ML-based Data Acquisition for Lateral Movement Detection in Science Networks

Diego Troy Lopez*, Thomas B. Reddington^{†§}, Robert Pahle*

*Research Technology Services, New York University, New York, United States of America

[†]Department of Computer Science, New York University, New York, United States of America

Email: {diego.lopez, tbr226, pahle}@nyu.edu

Abstract—Science DMZs are specialized networks designed for high performance scientific applications. Traditional security controls such as hardware firewalls are unsuitable in this context due to their throughput limitations. Network Intrusion Detection Systems (NIDS) analyze traffic out-of-line through mirroring techniques, which offers security value without compromising performance. Traditional NIDS leverage Switchport Analyzer (SPAN) to mirror Internet-bound traffic to the NIDS at the border of a Science DMZ. Capturing lateral movement traffic after an intrusion is challenging due to the cost of NIDS and physical connection requirements. While Encapsulated Remote SPAN allows mirroring traffic over IP, it is often seen as impractical due to the challenge of controlling its bandwidth usage. This work leverages Deep Learning techniques to develop an intelligent, dynamic NIDS capable of real-time lateral movement detection while preserving network performance. Our proposed system combines SPAN for mirroring Internet-bound traffic with filtered ERSpan for selective East-West traffic mirroring. An Autoencoder detects anomalous behavior by comparing traffic flows to learned normal patterns. When reconstruction errors exceed a threshold, the system dynamically adjusts access control lists to mirror East-West traffic from only these potentially compromised hosts. In a functional evaluation, the system successfully detected an attack chain and captured intra-VLAN lateral movement. During an operational evaluation in a production Science DMZ environment, the system demonstrated minimal control plane overhead with an acceptable bandwidth usage. This first-of-a-kind approach enhances security, preserves operational integrity, and yields significant cost savings over traditional NIDS deploys that provide similar coverage.

Index Terms—Science DMZ, Network Security, Computer Networks, Network Intrusion Detection, Machine Learning

I. INTRODUCTION

Science DMZs are a recent design pattern for Research & Education (R&E) computer networks supporting data-intensive scientific applications [1]. As opposed to enterprise networks which serve many business objectives, Science DMZs operate to maximize performance metrics such as throughput, latency, low packet loss, etc. These applications include data flows that extend to Terabyte or Petabyte scale, including portable DNA sequencing devices or facilities such as the Large Hadron Collider [2].

Security of Science DMZs is of utmost importance due to the sensitive nature of the scientific data traversing them. However, traditional in-line network security instruments such as firewalls or Network Intrusion Prevention Systems (NIPS)

impose latency and performance bottlenecks that make them unusable in the context of high performance networking [1] [3] [4]. Therefore, alternative security tools to secure Confidentiality, Integrity, Availability, and Performance [1] must be employed without adding additional latency or throughput constraints. Network Intrusion Detection Systems (NIDS) operate out-of-line and are well suited for this task [4].

NIDS analyzes a copy of network traffic out-of-line through packet mirroring and focuses on detection of intrusions [5]. NIDS do not impose throughput or latency constraints on the data path of traffic, however are unable to prevent intrusions if detected. NIDS may perform signature based or anomaly based detection [6] [7]. Signature based NIDS match on known malicious traces and are efficient at finding known attacks, but are fundamentally unable to detect novel or zero-day attacks [6] [7]. Anomaly based NIDS create a baseline profile of network traffic and detect anomalies and deviations from the baseline [8]. Anomaly based NIDS are able to detect zero-day attacks, however may experience false positives [7] [8]¹. Utilizing Machine Learning techniques for Anomaly based NIDS is well studied in current research [6] [9] [10].

Traditional deployments of NIDS leverage Switchport Analyzer (SPAN) on routers to mirror traffic from designated Ethernet ports to a monitoring port [11]. SPAN-based packet capture requires layer-1 connectivity between the NIDS and a target switch, making full-coverage monitoring via SPAN cost-prohibitive in large or distributed networks. As a compromise, NIDS are commonly deployed only at strategic locations such as at institutional border routers where fewer uplinks capture a high volume of security-relevant traffic. An example of this deployment strategy for Science DMZs can be seen in 1. Encapsulated Remote SPAN (ERSpan) extends SPAN by mirroring traffic over IP networks via Generic Routing Encapsulation tunnels, enabling a single NIDS to monitor a fleet of switches remotely. ERSpan can be filtered and controlled in real-time via ACLs over REST API or gRPC Network Management Interface. While powerful, ERSpan has seen little mention in the literature or real-world deployments due

¹Anomaly-based NIDS does not inherently analyze or confirm whether an anomaly is an actual attack. The distinction between false positives and true positives is typically a downstream task that requires correlation with other logs and contextual information.

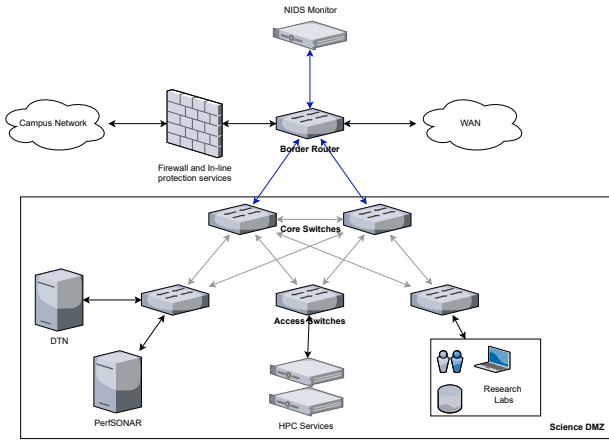


Fig. 1. Science DMZ co-located with Campus Network with NIDS monitoring. Links in blue indicate traffic mirrored to the NIDS.

to concerns over its potential bandwidth usage, which can overwhelm networks. Few projects have explored dynamic or programmatic control to mitigate these challenges and fully leverage its potential.

Substituting a firewall appliance at the border for a NIDS appliance to satisfy performance requirements necessarily means that it is possible to detect initial infections from the Internet, but less possible to prevent them. Simultaneously, the prohibitive cost of deploying NIDS requires operators to leave much if not all East-West traffic unmonitored. Lacking East-West monitoring coverage presents a critical visibility gap in the Science DMZ. Attackers who have weapons to compromise hosts in the Science DMZ are able to do so without worries of edge next-generation protection services preventing attacks, and moreover are able to move laterally within the network untraced. Without network logs as a ground truth, network operators experience great difficulties in identifying which hosts may have been compromised, as system logs may be altered by successful attacks.

To address the gap in ground-truth network logs of lateral movement post-infection in Science DMZs, we present an intelligent and dynamic NIDS which enhances perimeter monitoring deployments. Our proposed NIDS observes traffic over the fewer ISP links via SPAN, and uses Unsupervised Deep Learning techniques to generate anomaly scores of North-South flows in real time based on statistical deviations from a learned behavioral baseline. The system uses anomaly scores of North-South flows as a key indicator of compromise. Once scores for a host cross a operator-defined threshold, the NIDS issues API calls in real-time to alter ACLs governing ERSPAN tunnels, selectively mirroring East-West traffic of these hosts to the NIDS for collection and analysis.

Our NIDS leverages SPAN's inherent strengths for performant monitoring of a smaller number of high-priority links, while exploiting ERSPAN's capability to mirror East-West traffic throughout the entire network selectively. We utilize machine learning to intelligently identify and mirror traffic only from hosts that exhibit the most anomalous behavior in

real time. This capability allows the NIDS to float coverage dynamically across the network, adjusting visibility to be in the right place at the right time, without overwhelming available network bandwidth. We validate our approach on both functional and operational scenarios. This approach maintains Science DMZ operational integrity, while strengthening visibility of potential lateral movement while yielding significant cost savings over traditional NIDS deploys that provide similar coverage. To our knowledge, to date, this is the first approach introducing intelligence to ERSPAN filtering for East-West traffic monitoring.

II. BACKGROUND

A. Science Demilitarized Zone

The Science DMZ is a dedicated network enclave within an organization that is purpose-built for large science data transfers, with particular emphasis on high-throughput transfers over the Wide Area Network [1]. This is accomplished using four sub-patterns. Implementing the Science DMZ in proper physical location, utilizing dedicated Data Transfer Nodes (DTNs), active performance monitoring with tools such as PerfSONAR, and proper security which does not compromise performance of the network [1] [2] [12].

Security of the Science DMZ must satisfy Performance, Confidentiality, Integrity and Availability [1]. Notably, if the performance of the Science DMZ is compromised, then the mission of the infrastructure is compromised [1]. Hardware firewall appliances, typically the first line of defense in network security, are not used as they cause major performance penalties. Advertised firewall bandwidth is typically not based on large data transfer (elephant flow traffic) patterns which are often seen in Science DMZs, and have much lower performance [13]. The Science DMZ can be secured in part by moving security controls associated with the hardware firewall to more performant mechanisms. Namely, this includes router ACLs, Network Intrusion Detection Systems (NIDS), and Host Intrusion Prevention Systems (HIPS) [4].

B. Science DMZ Threat Model

Networks such as Science DMZs at research universities can be categorized under the broader threat model for Higher Education (HE). Science DMZs do not support administrative, HR or teaching missions of HE institutions by design, and thus only research data, missions and infrastructure are considered in terms of attack surface or threat actors.

Threat actors targeting Science DMZs may include cyber-criminals, nation-state actors, opportunists and hactivists [14]. Cyber-criminals may compromise systems and use malware or ransomware for immediate financial gain. Nation State actors are sophisticated attackers who focus on stealing and ex-filtrating intellectual property or research data from HE institutions, as well as espionage [14] [15]. Opportunists are attackers who attempt to compromise systems for fun, or to test their knowledge, while Hacktivists may target availability of research they deem heretical.

Assets in Science DMZs include the broad category of research data, as well as supporting infrastructure. Research data may include scientific data, academic knowledge, analysis results, intellectual property or publications [14]. Research data in the Science DMZ is typically digital and extends to Terrabyte or Petabyte scale. Infrastructure in the Science DMZ include DTNs, Data Acquisition Nodes (DAQs), High Performance Computing clusters and domain-specific sensors or instruments.

C. Packet Capturing Techniques

Network logs represent a valuable source of information for analyzing application behavior, as well as investigating security incidents. Many methods are available, which come with trade-offs and design considerations.

Switchport Analyzer (SPAN) or Port Mirroring is a technique that allows for the mirroring of traffic on one or more source interfaces to a destination interface on the same network device. SPAN thus requires destination hosts receiving the traffic to be directly connected to the switch where the SPAN session is configured. SPAN is typically performant and programmed in-hardware on high end switches, which does not introduce significant overhead. However, SPAN does not scale because it requires physical connectivity. It is typically infeasible for organizations to place NIDS sensors at every switch where administrators intend to monitor.

Encapsulated Remote SPAN (ERSPAN) is a technique which mirrors packets seen on a network device to IP based destinations. This is typically done by encapsulating the target packet in GRE tunnels to the destination analysis host, which then decapsulates the traffic. While ERSPAN is flexible and scalable from the physical perspective, mirrored traffic must traverse uplinks and consume bandwidth along the path from the network device to the NIDS. As an example, if every uplink-bound packet is mirrored, the total uplink bandwidth would be cut by $\frac{1}{2}$. Network devices can use ACL instrumentation to selectively mirror packets to destinations over ERSPAN. Examples include selecting by source IP, destination IP, destination TCP port, etc. ERSPAN may be altered in real time via REST API. ERSPAN is often considered impractical due to the challenge of controlling the bandwidth usage, however few projects exist in the literature which explore programmatic filtering of ERSPAN.

Methods such as Netflow, IPFIX and sFlow can provide information on network flows directly from switching hardware [16] [17] [18]. These methods are trivial to deploy and provide insights, however lack detailed analysis that can be generated from tools such as Zeek, Snort or Suricata. These methods may also be sampled to avoid performance penalties, which compromise fidelity during investigations.

While network traffic can also be collected on the end-host themselves, this is typically not feasible in high performance environments as the load to process the traffic falls to the CPU, which may already be a significant bottleneck for application performance.

D. Network Intrusion Detection in the Science DMZ

Common security practice for Science DMZs relies on Network Intrusion Detection Systems (NIDS). NIDS differs from firewalls and other Network Intrusion Prevention Systems (NIPS) by analyzing traffic in real time, but out of line. Because NIDS analyze traffic out-of-line, these methods are able to preserve the performance of the Science DMZ while adding value to security efforts. NIDS can function as either signature based or anomaly based [7].

Signature based NIDS relies on matching traffic captures to known-malicious traces [7] [6]. Signature based NIDS is efficient at detecting known malicious attacks, but are unable to detect zero-day attacks or variants of known attacks [7] [6]. Anomaly based NIDS function by building a baseline “normal” traffic profile, and identifying deviations to this as potential intrusions [8]. Anomaly based NIDS uniquely can detect novel and zero-day attacks, however are prone to false positives [8] [7]. Anomaly based NIDS operate under the assumptions that intrusion activity is noticeably different from normal system activities, and that malicious traffic is a subset of anomalous traffic in a network [8].

A key component of the Science DMZ architecture is replacing the hardware firewall appliance commonly seen at the border with a NIDS appliance. This necessarily means that attacks originating from the Internet can be detected at the border, but not prevented. If an attacker has a weapon to compromise a host in the Science DMZ, they are able to do so without edge next-generation prevention techniques blocking the attack, and can further infect additional hosts in the network. Visibility and logging is thus critical to detecting lateral movement and compromise of systems in the Science DMZ. Network logs collected by NIDS appliances serve as a source of truth of whether two systems communicated, which aids in detection of lateral movement post-initial infection. Without network logs aiding in investigations, cyberinfrastructure operators may incur significant costs to identify compromised hosts as anti-forensic techniques such as log cleaning may be employed on compromised systems.

NIDS must be placed at a specific layer of the Science DMZ network architecture. Potential layers depend on exact implementation but may include the access layer, distribution layer, core layer, or at the border of the Science DMZ. Choosing the layer where NIDS will monitor is a balancing act between cost and coverage. A common location for NIDS is at the border of the Science DMZ, which efficiently captures the North-South traffic leaving through the relatively small number of links, however leaves all East-West traffic unmonitored. The farther away the NIDS monitoring is from the access layer, the lower the coverage of East-West traffic. However, the closer the NIDS monitoring is to the access layer, the more NIDS are required to have full coverage of the network, as the access layer is typically the most geographically dispersed, the largest total count of network devices, and the highest volume of traffic.

NIDS appliances deployed in Science DMZs must be able

to process the traffic load of the network, which is often 100-400 Gbps. Appliances rated for this throughput are typically cutting-edge and exceed tens of thousands of dollars. It is generally cost-prohibitive to deploy full-coverage NIDS at the access level in larger Science DMZs. While deployment at the distribution or core layer is more feasible, this is still challenging in large environments, and misses any access-layer lateral movement. As a result, operators typically deploy a smaller number of NIDS appliances for North-South monitoring at the edge of the Science DMZ only. These NIDS deployments in Science DMZs leave much of the East-West traffic unmonitored. Methods reducing traffic processed by NIDS appliances have been well studied, and include shunting [19].

E. Zeek Network Traffic Analysis

Zeek is an open-source and widely deployed traffic analyzer for network security monitoring [20]. Zeek consists of 2 major components, the Event Engine and Policy Script Interpreter. The Event Engine comprises a processing pipeline of various analysis plugins which process attributes at different layers. For example, the Packet Analysis plugin calls analyzers which extract information from low level protocols (Ethernet, MPLS, etc) until the IP layer is reached. At that point, a session internal to Zeek is created for the connection, and Dynamic Protocol Detection is employed to analyze the application layer protocol. The Policy Script Interpreter executes event handlers in the Zeek domain specific language (DSL) to execute actions such as alerting.

Zeek's logs consist of flows as opposed to packets, which represent details from the full bidirectional session. This provides additional context, such as if the connection terminated normally, if the connection attempt was rejected, total packets and bytes sent or received, etc. This feature alongside Zeek's multi-protocol analysis provides a comprehensive view of the traffic which is needed for downstream anomaly detection tasks.

Zeek is not natively multithreaded, however can leverage PF_RING, a high speed packet capture library for balancing traffic analysis across multiple Zeek processes [21]. PF_RING balances flows on a 5-tuple hash, where all flows of the same hash will be received by the same process. This is necessary to scale analysis to match the network load seen in Science DMZs.

F. Autoencoders for Anomaly Detection Tasks

Autoencoders are a type of unsupervised neural network which are trained to learn reconstructions which closely resemble the input data [22]. This is done via two components, an encoder and a decoder. The encoder maps the input data into a low-dimensional representation, and the decoder reconstructs the original feature vector from the encoded output. The encoding functions to bottleneck the network and force it to learn the patterns of the original data [23]. The deviation between the input data and the output is known as the reconstruction error. A encoder can be seen in equation (1) and a decoder in

(2) [22]. In these equations W is the weight, b is the bias, and σ is the activation function of the layer. The reconstruction error can be seen in (3).

$$h = \sigma(W_{xh}x + b_{xh}) \quad (1)$$

$$z = \sigma(W_{hx}h + b_{hx}) \quad (2)$$

$$\|x - z\| \quad (3)$$

Autoencoders consist of many layers, with each layer consisting of a set number of neurons. More layers and more neurons increases complexity of the model, which allows it to model more complex data distributions. If the model is not complex enough to model the data distributions, the model will underfit and accuracy will be poor. If the model is too complex to model the data distributions, the model may simply memorize the distributions and have poor accuracy over new samples. Architecture search for the best performing model is a complex challenge which involves careful tuning of hyperparameters.

Autoencoders are well studied for anomaly detection tasks. Autoencoders used for anomaly detection are trained to minimize the reconstruction error in normal data, where high reconstruction errors which exceed user-defined thresholds are indicators of anomalies [23]. This relies on the assumption that anomalies cannot be reconstructed accurately via low dimensional representations [24]. It is crucial that the normal data used for training is not contaminated.

Determining the reconstruction error threshold on what may be considered "anomalous" is an operator parameter which requires careful consideration. A high threshold may miss true positives while a low threshold may result in false positives. Thus, the threshold may be chosen by the operator's posture and risk appetite.

III. SYSTEM ARCHITECTURE AND DESIGN

A. Motivation

Our system design is motivated by the need for performant and cost effective capture of East-West traffic for lateral movement detection in Science DMZs. Deployment of NIDS which use SPAN for packet mirroring require direct physical connectivity, making this too costly in large Science DMZs. ERSPAN is used to capture traffic remotely, however must be reduced to not overload available bandwidth. While static filters may be used to reduce ERSPAN traffic, these lead to deterministic gaps in monitoring which may be exploited by attackers². The proposed system must be dynamic in nature, being able to both detect attacker movement and mirror traffic over ERSPAN from only the most high priority hosts, while filtering traffic from lower priority hosts to control bandwidth usage.

²An example may be a static filter to mirror all traffic with destination port 22 via ERSPAN to a NIDS appliance. An attacker may avoid detection by attacking a service on port 23.

B. System Overview and Design Goals

The primary objective of the system is to augment perimeter NIDS deployments via dynamic collection of East-West traffic from anomalous-behaving hosts for lateral movement detection. The key components of the system include:

- Conventional PF_RING + Zeek NIDS deployment with SPAN packet capture of North-South uplink traffic.
- A real-time deep learning pipeline for generating anomaly scores of North-South flows from processed Zeek data, assigning aggregated scores to hosts via time series statistical methods.
- ACL management which applies user-defined thresholds to selectively mirror East-West traffic of anomalous-behaving hosts via ACLs on the switches governing ERSpan tunnels. Requests are made via REST API.
- Monitoring of NIDS operational metrics, system process metrics, and network device control plane metrics via the Prometheus framework.

This architecture offers several advantages over current NIDS solutions. By mirroring East-West traffic with ERSpan, the system attains high fidelity lateral movement monitoring without requiring full-scale SPAN deployments throughout the network, which has prohibitive hardware costs. The system maintains the operational integrity of the Science DMZ by applying filters to ERSpan tunnels to only mirror traffic from hosts which are deemed anomalous. The operator is able to tune the detection threshold to balance sensitivity against bandwidth overhead. Moreover, the system modifies filters dynamically and in real-time using the intelligence gained from anomaly scoring. The dynamic approach also avoids deterministic monitoring gaps which would arise from static, one-time ERSpan filters. The architecture implementation is described in the following subsections.

C. Hardware and Software Configuration

The NIDS appliance should be a high performance general-purpose server, equipped with a maximum capacity dual-port network card which connects into the Science DMZ. One interface is used as the destination for the SPAN session mirroring North-South uplinks, while the second interface is configured with an IP address and is used exclusively for receiving ERSpan traffic. Separation of interfaces for SPAN and ERSpan destinations allows for trivial collection of bandwidth usage. A third interface may be used for device management, such as Secure Shell (SSH) access. The hardware evaluated is a Dell C4130 with a Mellanox ConnectX-6 VPI 2x100G network card. The NIDS runs Ubuntu 22.04.

Zeek is installed and compiled with PF_RING support. A single host Zeek cluster is configured with two workers, one for the network interface collecting SPAN traffic, and another for the network interface collecting ERSpan traffic. Numerous Zeek processes are defined per worker for load balancing. Zeek is configured to log connection information in JSON format. Zeek is set to process incoming data on the network interfaces in real time and produce various output logs in JSON format, including conn.log, http.log, ssl.log, etc.

D. Network Topology and Traffic Collection

The NIDS appliance is connected directly to the border router of the Science DMZ. A SPAN session is created which mirrors the North-South bidirectional traffic from the uplinks to the NIDS unfiltered at 100 Gbps.

ERSpan mirror sessions over GRE are configured at the access level of the network. The destination of the GRE tunnels are configured to be the IP of the dedicated ERSpan interface of the NIDS appliance. ERSpan is configured to be filtered through an IPv4 ACL by a class map. By default, a deny rule filters all traffic from being mirrored to the ERSpan destination. IPs must be explicitly added to the ACL to begin mirroring. The Filtered ERSpan session is configured to source across all dataplane ports as a service policy of the network device.

E. Adaptive Analysis and Response Engine

The following sections describe components of our application engine, implemented as a Python package. The various components are operated and maintained in-memory. A Prometheus client is started and set to expose process metrics such as CPU and memory usage.

F. Data Preprocessing

The Zeek conn.log output file is tailed, with each JSON object loaded into memory for processing sequentially and in real time. Any flows not in the IPv4 address family are dropped and not analyzed. Time series features, specifically five minute and ten minute counts of two-tuples (source and destination IP) and three-tuples (source and destination IP, destination port) are added as features and stored in memory. Categorical features, history, conn_state, and service are top-50 encoded from the relevant training datasets. Continuous features, namely orig_bytes, resp_bytes, orig_ip_bytes, resp_ip_bytes are linearly scaled to values between 0 and 1. In total, 160 features are used.

G. Anomaly Detection Module

A multilayer Autoencoder is defined and used to generate reconstruction errors of the flows in real time. After a flow is loaded into memory and processed via the specified transformations, the flow is scored by the model. Once the error is calculated, the resulting error is assigned to both the source and destination IP. The errors for each IP are maintained in memory as a moving average in a dictionary. The IP to reconstruction error mappings are exposed in real time via the Prometheus client.

The architecture implementation for the model selected during the evaluations is described in Section 4.1.

H. Threshold and API Module

For each new flow seen, the involved IP's moving average error is compared to an operator-configured threshold. The system can then enter one of four stages which can be seen in 1.

- If the target IP moving average reconstruction error exceeds the configured threshold and the IP is not part of the mirroring IPv4 ACL, then an access control entry is added allowing said IP. A REST API request is made to all participating switches permitting the IP in the ACL governing the ERSPAN tunnel filtering. This action begins mirroring East-West traffic for the IP across the fleet.
- If the target IP moving average reconstruction error falls below the configured threshold and the IP is part of the mirroring IPv4 ACL, then the access control entry permitting said IP is removed from the ACL. A REST API request is made to all participating switches removing the IP from the ACL governing the ERSPAN tunnel filtering. This action stops mirroring East-West traffic for the IP across the fleet.
- If the target IP's moving average reconstruction error exceeds the threshold and the ACL is active, no further action is taken.
- If the target IP's moving average reconstruction error is below the threshold and no ACL is active, no further action is taken.

Algorithm 1 Adaptive East-West ACL Management

Initialization:

```

0: Remove all existing ACLs on the target network devices.
0: Initialize ACLActiveMap  $\leftarrow \emptyset$  {Tracks if IP has active ACL}
0: Initialize ScoreMap  $\leftarrow \emptyset$  {Stores moving avg for each IP}
0: function PROCESSFLOW(flow)
0:   srcIP  $\leftarrow$  flow.srcIP
0:   dstIP  $\leftarrow$  flow.dstIP
0:   err  $\leftarrow$  ComputeMAE(flow)
0:   UpdateScore(srcIP, err) {Update moving average}
0:   UpdateScore(dstIP, err)
0:   for ip  $\in$  {srcIP, dstIP} do
0:     currMA  $\leftarrow$  ScoreMap[ip] {Retrieve moving average}
0:     if currMA > threshold and ip  $\notin$  ACLActiveMap
0:       then
0:         seqNum  $\leftarrow$  CreateACL(ip) {REST API call}
0:         ACLActiveMap[ip]  $\leftarrow$  seqNum
0:     else if currMA  $\leq$  threshold and ip  $\in$  ACLActiveMap
0:       then
0:         seqNum  $\leftarrow$  ACLActiveMap[ip]
0:         DeleteACL(ip, seqNum) {REST API call}
0:         ACLActiveMap  $\leftarrow$  ACLActiveMap  $\setminus$  {ip}
0:     end if
0:   end for
0: end function=0

```

To manipulate the network ACLs, HTTP requests are made to the network device's REST API endpoints with POST and DELETE methods. The associated sequence numbers are tracked in-memory by the engine. As the reconstruction scores are kept in-memory, upon startup of the script all active ACLs

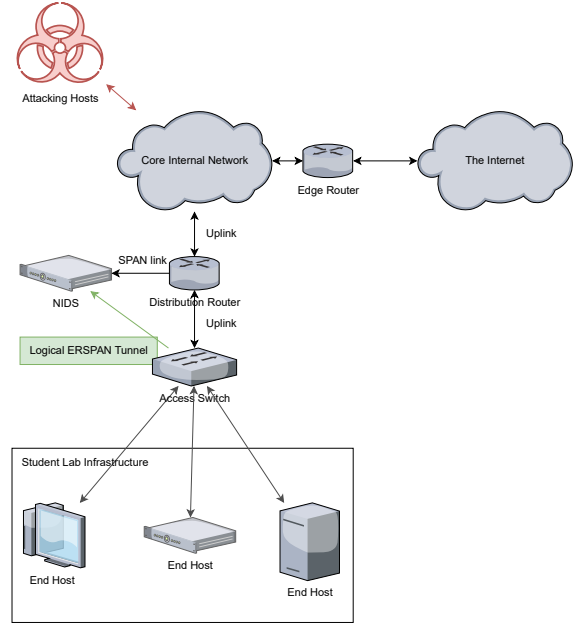


Fig. 2. Scenario Topology for Operational Evaluation.

on the target network devices are purged. Active ACLs are maintained in memory and exposed by the Prometheus client.

IV. FUNCTIONAL EVALUATION

To evaluate the system accuracy and function, a NIDS and the proposed system are deployed to detect an attack scenario performed on a non-production, ethereal lab infrastructure at a major U.S. university's Science DMZ. This infrastructure contains servers and workstations used for various testing and non-production use cases. The topology is seen in 2. The NIDS is deployed to monitor the North-South traffic at the distribution layer of the network. An ERSPAN tunnel is created from the access switch to the NIDS. North-South traffic consists of update traffic, remote management such as SSH, web browsing, and file transfers.

While the system is designed to analyze North-South traffic for the entire Science DMZ, we evaluate the system under a realistic attack by analyzing North-South traffic at the distribution level for this scenario. This is done so that systems are not attacked over external networks and systems. The access switch is a Dell PowerSwitch running Dell Enterprise SONiC version 4.2.0 (Edge), and the Distribution Router is a Dell PowerSwitch running Dell Enterprise SONiC version 4.2.0 (Standard).

A. Model Architecture Selection

A specific model architecture must be chosen for the Autoencoder which will optimize performance. This requires an evaluation of architectures on a dataset which includes both attacks and normal data. Our approach to model selection is to use a real dataset of 2 hours of known-normal Science DMZ traffic augmented with attack traffic from a labeled dataset

present in the literature. The dataset of normal Science DMZ traffic was created by collecting data at the border and cross-referencing with internal security data sources. Any traffic matching IPs which had security incidents in the last 1 month were removed.

We select the UWF-ZeekData22 dataset, which includes attack traffic from various MITRE ATT&CK categories [25]. The labeled attack data is combined with the known normal traffic, and the system transformations are done on the entire dataset. The dataset contained 12,044,735 samples of both normal and attack traffic.

Various Autoencoder architectures, including different layers, activation functions, and neurons are trained on the normal data. This included a search space of 1-5 layers per sub-model, with neurons ranging in a continuous decrease in the decoder while a continuous increase for the encoder. Neuron values were chosen from [2,4,8,16,24,48,64,80,96]. Latent space was chosen from [5,10,20]. Models are trained over 20 epochs with a batch size of 512.

A test set of 20% of the data is chosen for model evaluation. Mean Absolute Error (MAE) is chosen as the error criterion to reduce sensitivity to outliers as values may be extreme in Science DMZs. The reconstruction error one standard deviation above the median is chosen as a threshold to determine if a sample is classified as an anomaly. The best performing model was an Autoencoder, consisting of an Encoder with the architecture described in I, and a Decoder with the architecture described in II. The ReLU activation function is used in layers 1-7 while the sigmoid activation function is used in the final layer. The latent space of the autoencoder is a vector of length 20. The model had a total parameter count of 50920, of which 50104 was trainable.

TABLE I
ENCODER MODEL SUMMARY

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 96)	15 456
batch_normalization	(None, 96)	384
activation	(None, 96)	0
dense_1 (Dense)	(None, 64)	6 208
batch_normalization_1	(None, 64)	256
activation_1	(None, 64)	0
dense_2 (Dense)	(None, 48)	3 120
batch_normalization_2	(None, 48)	192
activation_2	(None, 48)	0
dense_3 (Dense)	(None, 20)	980
batch_normalization_3	(None, 20)	80
activation_3	(None, 20)	0

The model results are seen in III. The model achieved a base accuracy of 0.93, a Precision of 0.93, and a Recall of 0.92 at the supplied threshold. As seen in 3, the distribution of the attack traffic present in UWF-Zeekdata22 was easily separable from the normal traffic at a threshold of around 0.04 in the test set.

Given the scores from the model, the dataset was relatively “easy” from an anomaly detection perspective. The UWF-Zeekdata22 dataset consists of mostly reconnaissance attacks,

TABLE II
DECODER MODEL SUMMARY

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 20)	420
batch_normalization_4	(None, 20)	80
activation_4 (Activation)	(None, 20)	0
dense_5 (Dense)	(None, 64)	1 344
batch_normalization_5	(None, 64)	256
activation_5 (Activation)	(None, 64)	0
dense_6 (Dense)	(None, 96)	6 240
batch_normalization_6	(None, 96)	384
activation_6 (Activation)	(None, 96)	0
dense_7 (Dense)	(None, 160)	15 520

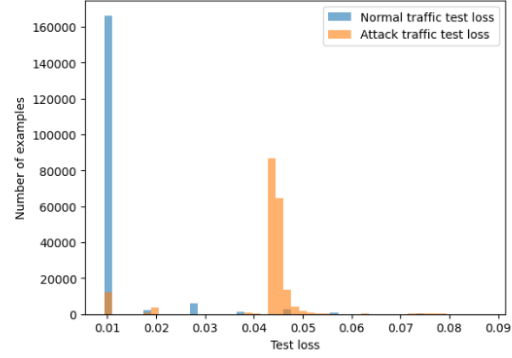


Fig. 3. Test Set Loss for Attack and Normal Traffic.

with minimal presence of other attack categories. Future work may seek to do further evaluations on future Zeek datasets which include realistic attack categories and tactics that may be seen in the Science DMZ.

B. Attack Chain Description

Attack chains are performed on hosts in the lab network to simulate remote-origin infection and lateral movement patterns. During this time, Zeek logs are scored in near real time, scores are aggregated and assigned to hosts, and East-West traffic is programmatically mirrored by the engine when crossing predefined thresholds. The NIDS is evaluated on the capability to detect incoming attacks in real time, and mirror east-west traffic within a single VLAN from potentially compromised hosts with minimal latency. The system’s performance is considered acceptable if it properly identifies incoming attack chains as anomalous, begins mirroring east-west traffic before lateral movement begins, and if the lateral movement traffic is captured and logged. Notably, the lateral movement performed in the scenario cannot be detected by traditional perimeter NIDS deployments. Normal lab traffic continues throughout the trials. The attack consists of the following characteristics.

- A host is scanned via nmap to determine open ports and potential vulnerabilities. The scan is performed as a SYN scan, with scan delay of 50 ms, T2 “polite” timing, across all ports.
- An SSH brute force attack is performed with Metasploit’s SSH login module using credentials present in the

TABLE III
MODEL RESULTS

Metric	Count
True Positive	166 388
True Negative	179 500
False Positive	12 253
False Negative	13 589

```
{
  "ts": 1744745823.196421,
  "uid": "CtBycA2zj3UqwXecx",
  "id.orig_h": [REDACTED],
  "id.orig_p": 42642,
  "id.resp_h": [REDACTED],
  "id.resp_p": 22,
  "auth_attempts": 0,
  "client": "SSH-2.0-OpenSSH_9.7p1 Ubuntu-7ubuntu4.2"
}
```

Fig. 4. Zeek SSH Log of Detected Lateral Movement.

rockyou-45.txt password file. BRUTEFORCE_SPEED is set to 1.

- Once the brute force completes and the attacking system accesses the host, the same credentials are used to move laterally to another host in the same VLAN over the SSH protocol. As the credentials are the same, the attack succeeds on the first attempt without reconnaissance or brute force.

The goal is to see if the attack traffic is correctly recognized as anomalous by the model, and if the system can accurately capture the east-west traffic post initial infection. The threshold for mirroring is set to the 90th percentile anomaly score of the training set. The attacks begin at a time marked 00:00 (MM:SS).

Latency was measured to determine the overhead time from when a flow is first read by the engine and before transformations or scoring, through to when the NIDS receives the 200 code from the API request confirming the ACL is active on the network device. The goal is to determine the overall latency from when the threshold-exceeding flow arrives, to when the system has acted and received confirmation from the network device.

C. Results

The anomaly scores for the IPs during the experiment is shown in 5. The attacked IP is shown in light green. The orange area showcases values above the 90th percentile anomaly score from the training set, while the red area showcases above the 99th percentile anomaly score. The anomaly score for the attacked IP crossed the 90th percentile threshold at time update 02:30, and the 99th percentile threshold at time 03:15 during the port scan. The anomaly score continued to increase until the end of the experiment. At no point during the attack did the anomaly score drop below the 99th percentile once it was crossed.

The lateral movement traffic initiated from the compromised host to the next hop was successfully mirrored and logged via

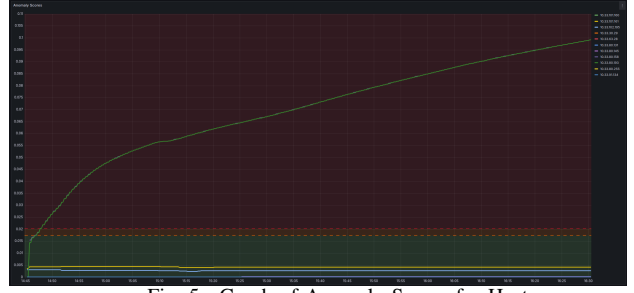


Fig. 5. Graph of Anomaly Scores for Hosts.

```
{
  "ts": 1744745823.195819,
  "uid": "CtBycA2zj3UqwXecx",
  "id.orig_h": [REDACTED],
  "id.orig_p": 42642,
  "id.resp_h": [REDACTED],
  "id.resp_p": 22,
  "proto": "tcp",
  "duration": 4.84253716468811,
  "orig_bytes": 6937,
  "resp_bytes": 0,
  "conn_state": "S0",
  "local_orig": true,
  "local_resp": true,
  "missed_bytes": 0,
  "history": "SAD",
  "orig_pkts": 97,
  "orig_ip_bytes": 10541,
  "resp_pkts": 0,
  "resp_ip_bytes": 0,
  "tunnel_parents": [
    "CNIXhhTmLc4obQwA4"
  ],
  "orig_l2_addr": [REDACTED],
  "resp_l2_addr": [REDACTED]
}
```

Fig. 6. Zeek CONN Log of Detected lateral movement.

Zeek. Logs for the attack include conn log and ssh logs. In total, the system mirrored 84.699 MB of East-West traffic from the host during the evaluation which included both malicious and normal traffic flows for the host. The latency observed for activating East-West mirroring for the given IP was 0.2934 seconds. The caught lateral movement ssh log can be seen in 4, and the caught lateral movement conn log can be seen in 6.

V. OPERATIONAL EVALUATION

To evaluate the system's operational performance and demonstrate capabilities, we deploy the proposed system on a dedicated NIDS appliance connected to the border router of a major U.S. university's implementation of the Science DMZ. As shown in 7, the NIDS uses SPAN-based mirroring of North-South traffic while leveraging dynamically filtered ERSpan for East-West flows within the Science DMZ. We evaluate the operational characteristics of the system over a 36 hour period for a subset of the network containing 5 production access switches.

During the first 12 hours (the "baseline" period), the NIDS monitors and processes North-South traffic via SPAN only, in the traditional perimeter NIDS architecture. In the next 24 hours (experiment phase) the NIDS anomaly detection

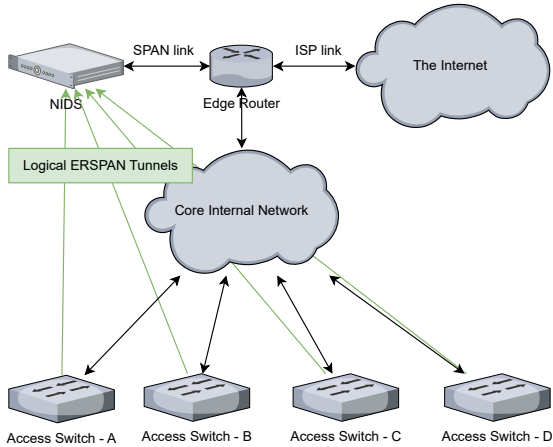


Fig. 7. Scenario Topology for Operational Evaluation.

architecture is activated, Zeek logs are scored in near real time, scores are aggregated and assigned to hosts, and East-West traffic is programmatically mirrored by the engine when crossing predefined thresholds. From hour 12 to 24, the 90th percentile reconstruction error of a clean dataset representing normal traffic is used as the threshold. During hour 24-36, a 99th percentile threshold is used to represent a lower vigilance posture.

Operational performance of the system is measured on additional NIDS appliance overhead for running the system, total ERSPAN bandwidth overhead at various thresholds, network device control plane overhead, and latency of the system. NIDS system performance is monitored via the Prometheus framework using Node_Exporter as well as the Prometheus client within the engine process. Network traffic, CPU Usage, and Memory Usage metrics are recorded. Raw metrics are recorded and statistical computations performed. The total volume of mirrored East-West traffic is measured by using counters on the NIDS dedicated network interface for ERSPAN. Network device CPU usage, memory, ACL counters and mirror session status are collected via REST API monitoring using the Telegraf package and recorded in Prometheus.

The same model from the functional evaluation is used for the operational scenario. We set the anomaly detection thresholds based on the reconstruction error distribution, using the 90th percentile (0.00429) and the 99th percentile (0.01583).

The Edge Router runs Arista EOS, while the access switches are Dell PowerSwitch running Dell Enterprise SONiC.

A. Results

During the 90th percentile trial 12-hour period, the node analyzed around 8.033 TB of traffic over the north-south link, while mirroring 301.43 GB of east-west traffic. The system mirrored traffic from 16 internal hosts. During the 99th percentile trial 12-hour period, the node analyzed 7.533 TB of traffic over the north-south link, while mirroring 36.066 GB of east-west traffic. The system mirrored east-west traffic from

12 internal hosts. The control plane statistics for the switches can be see in IV.

The process overhead for the engine is shown in V. The additional process used around 2.97 GB of memory during the 90th percentile trial and 3.07 GB of memory during the 99th percentile trial. The process used 44266 seconds of CPU time over a 12h period during the 90th percentile trial, and 44879.85 seconds of CPU time during the 99th percentile trial.

The average latency during the 90th percentile trail was 2.305 seconds, while the average latency during the 99th percentile trial was 4.804 seconds.

VI. DISCUSSION

The operational and functional experiments show that the system design is able to mirror East-West traffic from anomalous behaving hosts in real time, with minimal latency, in a responsive and accurate manner. The system was able to catch the attack chain and mirror the lateral movement flow to the NIDS which accomplished the objective. Notably, the system is able to mirror traffic within the same VLAN which did not pass through routing.

Bandwidth usage was successfully limited via thresholding to the most anomalous hosts in the network. The 99th percentile score significantly reduced the amount of bandwidth mirrored compared to the 90th percentile, however both were acceptable and did not saturate available network bandwidth. While the bandwidth usage was minimal during the trials, the exact usage in other Science DMZs will depend on the application traffic patterns present in the network as well as how much bandwidth is available for additional services.

Control plane overhead for network devices with ERSPAN mirror sessions was present, however negligible and within acceptable operating conditions. CPU usage increased by less than one percent. Memory usage was also minimal. Exact overhead will also depend on vendor implementation and if ERSPAN tunnels and ACLs are managed in hardware, or in software.

Process overhead for the engine on the NIDS was present however acceptable at 3 GB of memory and 44K seconds over a 12 hour period during the operational evaluation. The process CPU seconds used align with 100% usage of a single processor over the proposed time period. The 100% CPU usage of a single core represents additional load beyond the typical CPU usage for Zeek. This usage is likely due to the high overhead of running the Autoencoder via the CPU, due to a lack of GPUs. Full CPU usage likely contributed to increased latency during periods where Zeek logged many connections.

Latency was minimal and in the low seconds upon detection of an attack. The mirroring began during the reconnaissance phase of the attack, which captured the entire resulting attack chain. The latency observed is acceptable for capturing interactive attack chains targeting systems however may not be sufficient for automated attacks.

Instead of mirroring traffic via altering ACLs which govern ERSPAN mirroring, it is equally possible to alter dataplane ACLs to block lateral movement or isolate hosts entirely.

TABLE IV
SWITCH PERFORMANCE METRICS

Switch	Avg 12h Mem Usage	Avg 12h Memory Usage (Trial)	Total Mem Available	Avg 12h CPU Usage	Avg 12h CPU (Trial)
Switch A	2935 718	2971 577	3 648 588	35.5	35.5
Switch B	2457 603	2459 256	3 648 588	36.2	36.6
Switch C	2707 642	2740 956	3 648 588	37.8	37.9
Switch D	2473 785	2475 432	3 648 588	35.1	35.4

TABLE V
PROCESS CPU AND MEMORY USAGE BY PERCENTILE

Trial	Process CPU seconds	Process Resident Memory (Bytes)
90th PCT	44 266	2 973 462 528
99th PCT	44 879.85	3 070 300 160

If the operator wishes to pursue this, the system should be approached with extreme caution. Having anomaly based systems shut down hosts and ports has potential for serious service disruption and may be abused by attackers to perform Denial of Service (DoS) attacks. By mirroring traffic instead of shutting down hosts, the system has lower impact if potentially abused. Attacks on the system are detailed further in VII.

Notably, implementation of the system is done entirely through open source software, and the capabilities had no additional licensing or software costs on the hardware tested. This is relevant as it is accessible to smaller operators who wish to improve the security of their Science DMZs.

VII. LIMITATIONS

The Zeek monitoring software is inherently a “flow” based tool. Zeek analyzes an entire flow in totality to capture the entire context of the connection. Because of this however, Zeek does not log a flow until the connection is complete. This leaves Zeek open to attacks which deliberately leave connections open, as this would delay or block the logging from Zeek. Plugins such as `zeek_long_connections` exist which periodically log open connection information and should be enabled as a defense to this attack. If attacks which explicitly keep connections open are a considerable threat, different high performance NIDS tools which operate on a packet basis should be considered instead of Zeek. Examples include Snort and Suricata. If different tools are used, the feature set in which the model operates on would change, however the overall system design would remain the same. The analysis and model evaluation process was limited by the availability of labeled Zeek data. Many of the available network intrusion detection datasets are increasingly dated or use different tools. Many are also environment specific, such as targeting IoT. Obtaining a high quality network intrusion detection dataset with labeled Zeek data remains an open research challenge.

While Zeek may be used to generate logs from North-South or East-West traffic, only the North-South traffic is passed through to the system for anomaly scoring. The training set used only contains North-South traffic, which has fundamentally different patterns compared to East-West traffic. For this

reason it is not advisable to have the model infer anomaly scores of East-West traffic.

While it may be possible to augment the training set with East-West traffic theoretically, obtaining a training set of East-West traffic is extremely difficult and costly due to the high volume and difficulty of collection. One approach might be to have a “baseline period” where the system mirrors East-West traffic from a subset of the hosts in the network as a training set.

Because the system is not actively inferring on East-West traffic received, the system is not able to catch multiple hop lateral movement, or “chase” attackers beyond the first hop. The system will only mirror East-West traffic if the host’s North-South traffic is anomalous, feedback is not generated from the collected East-West traffic. Extensions to implement multihop lateral movement detection are discussed in Section 6.3.

Attackers may abuse the system and cause excessive mirroring by deliberately sending anomalous flows to a large number of hosts at the same time. Depending on the East-West traffic patterns at the time, this may cause a significant amount of traffic to be mirrored in a short duration, which has the potential for denial of service. System operators should pay careful attention to when IPs begin to have traffic mirrored, and the bandwidth usage of the system. One solution may be having a bandwidth target, and is detailed further in Section 6.3.

Latency of the system is a critical parameter and steps should be taken to reduce it to minimal levels. Latency is introduced by the individual components of the system, including SPAN, Zeek processing, system data processing, ML inference, and network latency of the REST API calls. This work did not pinpoint the exact source of the latency. We infer that the data processing and ML inference latency may be improved through more performant tooling, while Zeek and network latency may be more difficult to improve.

The system’s feedback is initiated via anomalous North-South traffic. This system does not cover attacks, such as insider threats or physical intrusions, where lateral movement may originate within the network.

VIII. FUTURE WORK

The system in its current state minimizes bandwidth usage by mirroring traffic from only the most anomalous hosts. Future work may explore assigning a bandwidth target, and optimizing between maintaining acceptable bandwidth usage and capturing traffic from anomalous hosts. For example, if

a highly anomalous system is mirroring traffic which starts to exceed a bandwidth target, the system may dynamically remove mirroring from other less anomalous hosts as an optimization. This may allow operators to set specific policy based on predefined or even current network utilization to maximize efficiency of network and security functions. The bandwidth feedback mechanism would also assist in preventing abuse by attackers who deliberately force the system into mirroring traffic from many hosts as part of a Denial of Service attack.

Extending the system to support multi-hop “chasing” of attackers as lateral movement continues beyond the first hop can greatly improve capabilities of the system. The most straightforward manner this can be accomplished is by recording a training dataset which includes East-West traffic and allowing the system to process these inputs in the same fashion as North-South traffic. Capturing an accurate representation of East-West traffic is difficult due to the high volume nature of the traffic. Another avenue may be a policy-based approach. If the system detects a highly anomalous flow which causes a host’s score to cross the threshold, the system may opt to mirror all traffic in the subnet across the specific destination port associated with the high anomaly score (if applicable). However, this solution may struggle with bandwidth usage if the port being attacked is also associated with high traffic volume. Operators who run tunable Host Intrusion Prevention Systems may also be able to increase sensitivity to attacks across services who’s ports have seen anomalous activity in the network.

The system only operates by analyzing conn logs, however may be extended to include anomaly detection for all of Zeek’s generated data streams. These include http, dns, ssh, ssl, etc. Adding application-based data streams will allow for more complex attacks to be detected. However due to the already 100% usage of a CPU thread for analyzing conn logs, it will be necessary to re-architect the system to be multi-processed or in a more performant framework.

A stress test should be conducted to see the impact of SPAN and ERSPAN mirroring on the control plane infrastructure of the network devices. This may include tests up to 100% of the line rate of port speed. CPU cycles should be monitored carefully on the network devices as this may lead to catastrophic control plane failures.

While this system leveraged Autoencoders as the model architecture, modern architectures such as RNN-based systems and Graph Neural Networks should be explored to capture attacks across spatial and temporal dimensions.

REFERENCES

- [1] E. Dart *et al.*, “The science dmz: A network design pattern for data-intensive science,” in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, 2013.
- [2] J. Crichigno, E. Bou-Harb, and N. Ghani, “A comprehensive tutorial on science dmz,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 2041–2078, 2018.
- [3] Q. Gong, P. DeMar, and M. Altunay, “Thundersecure: Deploying real-time intrusion detection for 100g research networks by leveraging stream-based features and one-class classification network,” *International Journal of Information Security*, vol. 21, no. 4, pp. 799–812, 2022.
- [4] I. Abhinit *et al.*, “Science dmz: Secure high performance data transfer,” 2022.
- [5] A. Khraisat *et al.*, “Survey of intrusion detection systems: techniques, datasets and challenges,” *Cybersecurity*, vol. 2, no. 1, pp. 1–22, 2019.
- [6] A. Javaid, Q. Niyaz, W. Sun, and M. Alam, “A deep learning approach for network intrusion detection system,” in *Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies (BIONETICS)*, 2016, pp. 21–26.
- [7] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, and K.-Y. Tung, “Intrusion detection system: A comprehensive review,” *Journal of Network and Computer Applications*, vol. 36, no. 1, pp. 16–24, 2013.
- [8] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, “Network anomaly detection: Methods, systems and tools,” *IEEE Communications Surveys & Tutorials*, vol. 16, no. 1, pp. 303–336, First Quarter 2014.
- [9] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, Oakland, CA, USA, May 2010, pp. 305–316.
- [10] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, “A deep learning approach to network intrusion detection,” *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 2, no. 1, pp. 41–50, Jan. 2018.
- [11] J. Svoboda, I. Ghafir, and V. Prenosil, “Network monitoring approaches: An overview,” *International Journal of Advanced Computer Networking and Security*, vol. 5, no. 2, pp. 88–93, October 2015.
- [12] B. Tierney, J. Metzger, J. Boote, E. Boyd, A. Brown, R. Carlson, M. Zekauskas, J. Zurawski, M. Swany, and M. Grigoriev, “perfSONAR: Instantiating a global network measurement framework,” in *Proceedings of the SOSP Workshop on Real Overlays and Distributed Systems*, 2009, p. 28.
- [13] Cisco Systems, Inc., “Processing of single stream large sessions (elephant flow) by firepower services,” Cisco, Tech. Rep., April 2020, [Online]. Available: <https://hdl.handle.net/2022/27007>.
- [14] J. B. Ulven and G. Wangen, “A systematic review of cybersecurity risks in higher education,” *Future Internet*, vol. 13, no. 2, p. 39, Feb. 2021.
- [15] B. Potter, “Practical threat modeling,” *Login*, vol. 41, pp. 59–63, Fall 2016.
- [16] B. Claise, B. Trammell, and P. Aitken, “Specification of the ip flow information export (ipfix) protocol for the exchange of flow information,” IETF, RFC RFC7011, 2013.
- [17] B. Claise, “Cisco systems netflow services export version 9,” Cisco Systems, RFC RFC3954, 2004.
- [18] P. Phaal, S. Panchen, and N. McKee, “Inmon corporation’s sflow: A method for monitoring traffic in switched and routed networks,” InMon Corporation, RFC RFC3176, 2001.
- [19] V. Stoffer, A. Sharma, and J. Krous, “100g intrusion detection,” Lawrence Berkeley National Laboratory (LBL), Tech. Rep., 2015.
- [20] V. Paxson and R. Sommer, “The zeek network security monitor (bro),” <https://www.zeek.org/>, 1994, accessed: Nov. 5, 2024.
- [21] NTOP, *PF_RING User Guide*, NTOP, accessed: Nov. 13, 2024, [Online]. Available: https://www.ntop.org/guides/pf_ring/
- [22] J. An and S. Cho, “Variational autoencoder based anomaly detection using reconstruction probability,” *Special lecture on IE*, vol. 2, no. 1, pp. 1–18, 2015.
- [23] D. Gong, L. Liu, V. Le, B. Saha, M. R. Mansour, S. Venkatesh, and A. van den Hengel, “Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1705–1714.
- [24] B. Zong, Q. Song, M. R. Min, W. Cheng, C. Lumezanu, D. Cho, and H. Chen, “Deep autoencoding gaussian mixture model for unsupervised anomaly detection,” in *International Conference on Learning Representations*, 2018.
- [25] S. S. Bagui, D. Mink, S. C. Bagui, T. Ghosh, R. Plenkers, T. McElroy, S. Dulaney, and S. Shabanali, “Introducing UWF-ZeekData22: A Comprehensive Network Traffic Dataset Based on the MITRE ATT&CK Framework,” *Data*, vol. 8, no. 1, p. 18, Jan. 2023. [Online]. Available: <https://doi.org/10.3390/data8010018>